Automated MCQ Exam Portal

Course Project Report

By:

1. Zannatul Nayem — 0432320005101063
2. Reon Ahmed — 0432320005101066

Course Code: CSE 416
Course Title: Software Project Design and Development
Semester: Autumn 2026
Course Instructor: Dr. Mahfida Amjad Dipa
Section: 7B
Batch: 54

Department of CSE
University of Information Technology and Sciences (UITS)

Submitted in partial fulfillment of the requirements for the CSE 416 course project, according to the prescribed report format of the UITS Department of CSE.

Table of Contents

1. Introduction 3
1.1 Introduction 3
1.2 Motivation 3
1.3 Aims and Objectives 4
1.4 Scope and Limitations 4
1.5 Team Organization and Individual Contributions 4
2. Background Study and Requirement Analysis 4
2.1 Introduction 4
2.2 Related Work and Existing Solutions 4
2.3 Stakeholders and User Characteristics 5
2.4 Software Requirements Specification 5
2.5 Alternative Solutions and Feasibility 6

List of Tables

Table 2.1. Conceptual comparison of examination approaches 4
Table 2.2. Stakeholder characteristics and concerns 5
Table 2.3. Alternative approaches and feasibility considerations 6

No figures are included in this version; the listed visual content is limited to tables.

1. Introduction

1.1 Introduction

Automated MCQ Exam Portal is a web-based academic examination platform intended to support the preparation, delivery, evaluation, and review of multiple-choice examinations. The system is organized around three role types—Student, Teacher, and Admin—and provides a structured workflow from question-pool management to result publication.

The project addresses a real-life academic context in which examination activities may otherwise depend on manual coordination, paper-based records, delayed marking, and fragmented result management. Its proposed workflow uses timed examinations, automatic evaluation, role-aware access, analytics, and audit logging while keeping the system within the stated MCQ-focused scope.

1.2 Motivation

The motivation is to reduce repetitive manual work in examination preparation and marking, minimize the possibility of human grading errors, avoid unnecessary delay in result publication, improve the organization of examination records, and enable useful performance analytics. These are design goals rather than measured outcomes; no quantitative improvement is claimed without final verification evidence.

Design intent. The portal is intended to make academic MCQ examinations more consistent, traceable, and manageable while preserving appropriate human responsibility for question preparation, review, and academic decisions.

1.3 Aims and Objectives

- **Functional:** provide authentication, role separation, course and question-pool management, timed exams, automatic evaluation, result history, analytics, and audit logging.
- **Technical:** implement a maintainable React/TypeScript frontend, NestJS/TypeScript backend, Prisma data layer, Supabase PostgreSQL database, and JWT-based role-aware authentication.

- **Academic:** apply requirements analysis, modeling, software design, implementation, testing, deployment planning, documentation, and professional review practices.
- **Project:** coordinate a two-person development effort, maintain traceable responsibilities, integrate the system modules, and prepare a submission-ready report without overstating unavailable evidence.

1.4 Scope and Limitations

In scope are Student, Teacher, and Admin workflows; login and registration; course and question-bank management; exam creation; timed MCQ sessions; automatic submission and evaluation; result publication and history; analytics; notifications or support information; and audit-log review. The current focus is MCQ assessment rather than a general assessment suite.

The system depends on suitable internet connectivity and a compatible browser. Frontend production deployment is on Vercel, and the backend API is live on Render. The deployed architecture uses Vercel for the React frontend, Render for the NestJS API, and Supabase PostgreSQL for persistent data storage. The report does not claim unsupported proctoring, external LMS integration, formal accessibility certification, or other product-specific capabilities not supplied in the project facts.

1.5 Team Organization and Individual Contributions

The project is organized as a two-person team with shared accountability. Reon Ahmed is responsible for frontend/UI work, deployment setup, integration, project coordination, testing, and documentation. Zannatul Nayem is responsible for backend/API work, database and Prisma activities, examination and evaluation modules, testing, and documentation.

Both members participate in requirements review, design discussion, integration, peer review, testing, presentation preparation, and final verification. The responsibility areas describe ownership and coordination, not fabricated commit counts or unsupported claims about individual effort percentages.

2. Background Study and Requirement Analysis

2.1 Introduction

This chapter establishes the problem context, stakeholders, requirements, and feasibility basis for the Automated MCQ Exam Portal. The analysis is written as a project baseline: proposed or planned behavior is distinguished from implementation evidence that must be checked during final verification.

2.2 Related Work and Existing Solutions

The comparison below is conceptual and is based on the project requirements. It does not evaluate named products and does not introduce product-specific statistics.

Criterion	A. Traditional/manual exam process	B. Generic online form/quiz tools	C. Proposed Automated MCQ Exam Portal
Workflow	Paper or manually coordinated activities.	Form-centered workflow with limited academic tailoring.	Role-aware workflow covering preparation, delivery, evaluation, and review.
Timing	Often coordinated manually.	May provide basic time controls depending on configuration.	Timed MCQ examination session with planned automatic submission behavior.
Evaluation	Manual checking and marking.	Automatic checking may be available for configured quiz items.	Automatic evaluation designed around the portal's question and answer model.
Result management	Records may be distributed across papers or files.	Results depend on the tool's available reporting features.	Result publication, detailed results, and student result history are defined features.
Role separation	Often managed through institutional procedures.	May be limited or configured externally.	Student, Teacher, and Admin roles are explicit design entities.
Analytics	Requires manual compilation.	Basic summaries may be available.	Teacher-oriented performance review and analytics are included in scope.

Criterion	A. Traditional/manual exam process	B. Generic online form/quiz tools	C. Proposed Automated MCQ Exam Portal
Auditability	Depends on record-keeping discipline.	Depends on tool history and export facilities.	Audit logging is a stated system capability and verification concern.
Academic suitability	Familiar but labor-intensive.	Useful for simple quizzes but may not match the complete role and record requirements.	Designed specifically for the stated academic MCQ examination workflow.

Table 2.1. Conceptual comparison of examination approaches.

2.3 Stakeholders and User Characteristics

Stakeholder	Access and responsibility	Usability and security concerns
Student	Views available exams, reads instructions, takes exams, submits answers, views results, manages profile, and receives support information.	Clear navigation, timer visibility, understandable validation, privacy of results, and prevention of unauthorized access.
Teacher	Manages courses, question banks, exams, analytics, and student performance review.	Efficient content management, protection of question data, accurate evaluation, and readable summaries.
Admin	Manages users, courses, exams, and audit-log review.	Least-privilege administration, careful changes, accountability, and protection of sensitive records.
Course/project instructor	Reviews project scope, engineering process, documentation, demonstration, and evidence.	Traceability, honest reporting, readable artifacts, and clear distinction between planned and verified work.
Institutional/academic management	Potential consumer of organized examination records, results, and administrative oversight.	Reliability, record retention, role accountability, privacy, and operational suitability.

Table 2.2. Stakeholder characteristics and concerns.

2.4 Software Requirements Specification

Functional requirements. The following requirements define the intended system behavior and should be checked against the implementation during final verification.

- **Authentication and authorization:** support login and registration as applicable; authenticate users with JWT-based sessions; identify Student, Teacher, and Admin roles; and restrict role-sensitive operations.
- **Student operations:** view available exams; read instructions; enter a timed exam; answer MCQs; submit manually or through the defined timeout behavior; view detailed results and result history; manage profile information; and access notifications or support information.
- **Teacher operations:** manage courses; create and maintain question banks and question pools; create and manage exams; review exam lists; inspect student performance; and use analytics relevant to available results.
- **Admin operations:** manage users, courses, and exams; review audit logs; and perform administrative actions subject to authorization rules.
- **Examination and evaluation:** configure timed MCQ exams; maintain question and answer data; validate submissions; evaluate answers automatically according to the configured answer key; publish results; and retain result history.
- **Notifications/support:** provide the defined notification or support information pathways without implying an external messaging service unless separately verified.
- **Audit logging:** record appropriate security-sensitive or administrative events in a reviewable form, subject to the final implementation and privacy decisions.
- **Reporting and analytics:** present available results and teacher-oriented performance summaries without claiming statistical outputs that have not been generated and verified.

Non-functional requirements.

- **Usability:** workflows should be understandable, consistent, and readable for the three defined roles.

- **Performance:** ordinary navigation, exam interaction, and result retrieval should respond within acceptable project-defined expectations; measurements remain to be recorded if required.
- **Security:** apply role-aware authorization, secure secret handling, input validation, protected authentication data, least privilege, and careful error exposure.
- **Reliability:** preserve exam and result consistency, handle failures visibly, and avoid silent loss of submitted answers where the implementation supports recovery.
- **Maintainability:** keep frontend, backend, data access, and presentation responsibilities separated and documented.
- **Scalability:** use modular boundaries that can support additional courses, users, questions, and exams, subject to infrastructure limits.
- **Compatibility:** support a suitable modern browser and the project's web deployment environment; browser coverage should be recorded during final checks.
- **Auditability:** make relevant administrative and security events traceable while respecting privacy and retention decisions.

Assumptions and constraints. Users have internet and browser access; teachers provide valid question and answer data; the current assessment focus is MCQ; Supabase PostgreSQL is available to the configured data layer; frontend deployment on Vercel is available; backend deployment to Render is pending verification; and exact repository, environment, and execution evidence must be checked before submission.

Acceptance-oriented statements. A requirement should be considered ready for acceptance only when the responsible team member can demonstrate the behavior, record the test result, preserve relevant evidence, and obtain review. This report does not treat every requirement as formally verified.

2.5 Alternative Solutions and Feasibility

Alternative	Technical and operational view	Feasibility assessment
Manual examination	Low software dependency but high coordination, marking, and record-management effort.	Academically familiar, but poorly aligned with the automation and analytics objectives.

Alternative	Technical and operational view	Feasibility assessment
Generic form/quiz tooling	Can reduce setup effort, but role separation, auditability, workflow control, and academic tailoring may be limited.	Feasible for simple quizzes; less suitable for the complete stated requirement set.
Monolithic custom application	Can centralize functionality, but may make module boundaries, testing, and future maintenance less clear.	Technically possible, but less aligned with the selected separation of concerns.
Selected modular web application	Separates React/Vite presentation, NestJS services, Prisma data access, and Supabase PostgreSQL storage.	Feasible for the course project and appropriate for iterative implementation, subject to final integration and deployment verification.

Table 2.3. Alternative approaches and feasibility considerations.

The selected stack balances technical feasibility, course learning objectives, maintainability, and the need for role-aware academic workflows. No unsupported cost or productivity claim is made; economic feasibility should be assessed through available course resources and the final deployment configuration.